다항식 연산과 DFT 분석

개요

이 문서는 Plonky3에서 구현된 다항식 연산과 이산 푸리에 변환(DFT) 알고리즘들을 분석합니다. Plonky3는 STARK 증명 시스템의 핵심 구성 요소인 다항식 연산을 효율적으로 처리하기 위해 여러 종류의 FFT 알고리즘과 보간 기법을 제공합니다.

주요 구성 요소:

- **p3-dft**: 2-adic 부분군에서의 다양한 FFT 구현
- **p3-interpolation**: Lagrange 보간법 구현
- **p3-circle**: Circle group FFT (Mersenne31 전용)

수학적 배경

이산 푸리에 변환 (DFT)

DFT는 다항식의 계수 표현과 평가 표현 사이의 변환을 수행합니다. 크기 N인 2-adic 부분군 H에서:

```
DFT: 계수 (c₀, c₁ , ..., c_{N-1}) → 평가 (f(ω⁰), f(ω¹), ..., f(ω^{N-1}))
```

여기서 ω는 N차 단위근이고, $f(x) = \Sigma_i c_i x^i$입니다.

Coset DFT

일반적인 부분군 대신 coset gH에서의 평가를 계산합니다:

```
Coset DFT: 계수 → (f(gω⁰), f(gω¹), ..., f(gω^{N-1}))
```

이는 계수에 g^i를 곱한 후 일반 DFT를 적용하여 구현됩니다.

Low-Degree Extension (LDE)

작은 부분군에서 평가된 다항식을 더 큰 부분군으로 확장하는 과정:

1. 평가 → 계수 (IDFT)
2. 계수를 0으로 패딩
3. 계수 → 평가 (DFT)

DFT 구현체 분석

1. Radix-2 DIT (Decimation-in-Time)

파일: `dft/src/radix_2_dit.rs`

특징

- 전통적인 Cooley-Tukey FFT 알고리즘
- 입력을 bit-reverse 순서로 재배열
- 각 레이어에서 butterfly 연산 수행
- Twiddle factor 메모이제이션

구현 세부사항

```
pub struct Radix2Dit<F: TwoAdicField> {
twiddles: RefCell<BTreeMap<usize, Vec<F>>>,
}
```

알고리즘 흐름:

1. 입력 행렬을 bit-reverse 순서로 재배열
2. $\log_2(N)$ 개의 레이어에서 butterfly 연산 수행
3. 각 레이어에서 twiddle factor 적용

Butterfly 연산:

```
fn dit_layer<F: Field>(mat: &mut RowMajorMatrixViewMut<'_, F>, layer: usize, tw
let half_block_size = 1 << layer;
let block_size = half_block_size * 2;
mat.par_row_chunks_exact_mut(block_size)
.for_each(|mut block_chunks| {
let (mut hi_chunks, mut lo_chunks) = block_chunks.split_rows_mut(half_block_si
// Butterfly 연산 적용
});
}
```

성능 특성:

- 시간 복잡도: O(N log N)
- 공간 복잡도: O(N)
- 배치 처리에 최적화
- 중간 크기 변환에 적합

2. Radix-2 Bowers FFT

파일: `dft/src/radix_2_bowers.rs`

특징

- "Improved Twiddle Access for Fast Fourier Transforms" 논문 기반
- 메모리 접근 패턴 최적화
- Twiddle factor 접근 개선
- G 네트워크와 G^T 네트워크 구현

구현 세부사항

Bowers G 네트워크는 입력이 bit-reverse 순서라고 가정하고 DFT와 유사한 연산을 수행합니다:

```
fn bowers_g<F: TwoAdicField>(mat: &mut RowMajorMatrixViewMut<F>) {
let h = mat.height();
let log_h = log2_strict_usize(h);

let root = F::two_adic_generator(log_h);
let mut twiddles = root.powers().collect_n(h / 2);
reverse_slice_index_bits(&mut twiddles);
for log_half_block_size in 0..log_h {
butterfly_layer(mat, 1 << log_half_block_size, &twiddles)
}
}
```

최적화된 Coset LDE:

```
fn coset_lde_batch(&self, mut mat: RowMajorMatrix<F>, added_bits: usize, shift
// G^T 네트워크 적용
```

```
bowers_g_t(&mut mat.as_view_mut());

// 계수 재스케일링 (1/height와 coset shift 적용)

let h_inv_subfield = F::PrimeSubfield::ONE.div_2exp_u64(log_h as u64);

let h_inv = F::from_prime_subfield(h_inv_subfield);

let mut weights = shift.shifted_powers(h_inv).collect_n(h);

reverse_slice_index_bits(&mut weights);

// 병렬 가중치 적용

mat.par_rows_mut()

.zip(weights.into_par_iter())

.for_each(|(row, weight)| row.iter_mut().for_each(|elem| *elem *= weight));

// Zero padding 및 G 네트워크 적용

mat = mat.bit_reversed_zero_pad(added_bits);

bowers_g(&mut mat.as_view_mut());

mat

}
```

성능 특성:

- 메모리 접근 패턴 최적화
- 캐시 효율성 향상
- LDE 연산에서 특히 효율적
- 대용량 변환에 적합

3. Radix-2 DIT Parallel

파일: `dft/src/radix_2_dit_parallel.rs`

특징

- 병렬 처리에 최적화된 FFT
- Butterfly 네트워크를 두 부분으로 분할
- 스레드 간 통신 최소화
- 복잡한 메모리 관리

구현 세부사항

두 단계 접근법:

1. **첫 번째 절반**: 작은 블록으로 DIT 수행
2. **두 번째 절반**: Bit-reverse 순서로 계속 진행

```
fn dft_batch(&self, mut mat: RowMajorMatrix<F>) -> Self::Evaluations {
let h = mat.height();
let log_h = log2_strict_usize(h);
let mid = log_h.div_ceil(2);

// 첫 번째 절반: 일반적인 DIT
reverse_matrix_index_bits(&mut mat);
first_half(&mut mat, mid, &twiddles.twiddles);

// 두 번째 절반: bit-reverse 순서로 DIT 뒤집기
reverse_matrix_index_bits(&mut mat);
second_half(&mut mat, mid, &twiddles.bitrev_twiddles, None);
```

```
mat.bit_reverse_rows()
}
```

Coset LDE 최적화:

- Out-of-place 연산으로 메모리 효율성 향상
- 여러 coset을 병렬로 처리
- 메모리 할당 최적화

성능 특성:

- 높은 병렬성
- 대용량 배치 처리에 최적
- 메모리 사용량 증가
- 복잡한 구현

4. Circle Group FFT

파일: `circle/src/cfft.rs`

특징

- Mersenne31 필드 전용 최적화
- Circle group 기반 다항식 평가
- 복소수 확장 필드 활용
- 특별한 기하학적 구조 이용

수학적 배경

Circle group은 복소수 단위원 위의 점들로 구성됩니다. Mersenne31에서는 $i^2 = -1$인 복소수 확장을 사용하여 circle을 정의합니다.

Point 구조:

```
pub struct Point<F> {
pub x: F,
pub y: F,
}
```

Circle 위의 점 (x, y)는 $x^2 + y^2 = 1$을 만족합니다.

구현 세부사항

Circle 기저 함수:

```
pub fn circle_basis<F: Field>(p: Point<F>, log_n: usize) -> Vec<F> {
let mut b = vec![F::ONE, p.y];
let mut x = p.x;
for _ in 0..(log_n - 1) {
for i in 0..b.len() {
b.push(b[i] * x);
}
x = x.square().double() - F::ONE; // Chebyshev 배가 공식
}
b
}
```

Twiddle 계산:

```
fn compute_twiddles<F: ComplexExtendable>(domain: CircleDomain<F>) -> Vec<Vec<F
let mut pts = domain.coset0().collect_vec();
reverse_slice_index_bits(&mut pts);
let mut twiddles = vec![pts.iter().map(|p| p.y).collect_vec()];
if domain.log_n >= 2 {
twiddles.push(pts.iter().step_by(2).map(|p| p.x).collect_vec());
for i in 0..(domain.log_n - 2) {
let prev = twiddles.last().unwrap();
let cur = prev
.iter()
.step_by(2)
.map(|x| x.square().double() - F::ONE) // Chebyshev 다항식
.collect_vec();
twiddles.push(cur);
}
}
twiddles
}
```

성능 특성:

- Mersenne31에서 매우 효율적

- 특별한 수학적 구조 활용
- 메모리 접근 패턴 최적화
- 제한된 필드 지원

보간 (Interpolation) 분석

Barycentric Lagrange 보간

파일: `interpolation/src/lib.rs`

수학적 원리

주어진 coset gH에서의 다항식 평가값으로부터 임의의 점 z에서의 값을 계산합니다:

```
f(z) = (z^N - g^N)/(N * g^N) * Σᵢ (gh^i)/(z - gh^i) * f(gh^i)
```

구현 세부사항

기본 보간 함수:

```rust
pub fn interpolate_coset<F, EF, Mat>(
coset_evals: &Mat,
shift: F,
point: EF
) -> Vec<EF>
where
F: TwoAdicField,
EF: ExtensionField<F>,
```

```
Mat: Matrix<F>,

{

let height = coset_evals.height();

let log_height = log2_strict_usize(height);



let coset = TwoAdicMultiplicativeCoset::new(shift, log_height)

.unwrap()

.iter()

.collect();



// 1/(z - gh^i) 계산

let diffs: Vec<_> = coset.par_iter().map(|&g| point - g).collect();

let diff_invs = batch_multiplicative_inverse(&diffs);



interpolate_coset_with_precomputation(coset_evals, shift, point, &coset, &diff_i

}
```

최적화된 보간:

```
pub fn interpolate_coset_with_precomputation<F, EF, Mat>(

coset_evals: &Mat,

shift: F,

point: EF,

coset: &[F],
```

```
diff_invs: &[EF],
) -> Vec<EF> {
// gh^i/(z - gh^i) 계산
let col_scale: Vec<_> = coset
.par_iter()
.zip(diff_invs)
.map(|(&sg, &diff_inv)| diff_inv * sg)
.collect();

let point_pow_height = point.exp_power_of_2(log_height);
let shift_pow_height = shift.exp_power_of_2(log_height);

// 소멸 다항식: Z_{sH}(z) = z^N - g^N
let vanishing_polynomial = point_pow_height - shift_pow_height;

// 스케일링 인수: Z_{sH}(z)/(N * g^N)
let denominator = shift_pow_height.mul_2exp_u64(log_height as u64);
let scaling_factor = vanishing_polynomial * denominator.inverse();

// 각 열 다항식에 대해 내적 계산 후 스케일링
let mut evals = coset_evals.columnwise_dot_product(&col_scale);
scale_slice_in_place_single_core(&mut evals, scaling_factor);
evals
```

```
}
```

성능 특성:

- 배치 역원 계산으로 최적화
- 사전 계산된 값 재사용 가능
- 병렬 처리 지원
- 확장 필드에서 효율적

병렬 처리 최적화

maybe-rayon 통합

Plonky3는 조건부 병렬 처리를 위해 `p3-maybe-rayon` 을 사용합니다:

```
use p3_maybe_rayon::prelude::*;


// 조건부 병렬 처리
mat.par_row_chunks_exact_mut(block_size)
.for_each(|mut  block| {
// 병렬 처리 로직
});
```

병렬화 전략

1. **데이터 병렬성**: 행렬의 행/열을 병렬로 처리
2. **작업 분할**: 큰 변환을 작은 블록으로 분할

3. **메모리 지역성**: 캐시 친화적인 접근 패턴

4. **부하 균형**: 작업량을 균등하게 분배

성능 고려사항

블록 크기 선택:

```
fn desired_num_jobs() -> usize {
16 * current_num_threads()
}

let job_sz = core::cmp::max(1, lo.len() >> log2_ceil_usize(desired_num_jobs()))
```

메모리 접근 패턴:

- Sequential access 최적화
- Cache line 정렬
- False sharing 방지

벤치마크 분석

FFT 성능 비교

테스트 환경 (dft/benches/fft.rs):

- 배치 크기: 256
- 로그 크기: 14, 16, 18, 20, 22
- 다양한 필드: BabyBear, Goldilocks, Mersenne31

주요 결과:

1. **Radix2DitParallel**: 대용량 변환에서 최고 성능
2. **Radix2Bowers**: 메모리 효율성과 성능의 균형
3. **Radix2Dit**: 중간 크기에서 안정적 성능
4. **Circle FFT**: Mersenne31에서 특화된 성능

LDE 성능 비교

Circle vs Two-adic (circle/benches/cfft.rs):

- Circle FFT가 Mersenne31에서 우수한 성능
- Two-adic FFT가 일반적인 필드에서 더 범용적

실제 사용 예제

기본 DFT 사용

```
use p3_dft::{Radix2Dit, TwoAdicSubgroupDft};
use p3_baby_bear::BabyBear;
use p3_matrix::dense::RowMajorMatrix;

let dft = Radix2Dit::<BabyBear>::default();
let coeffs = vec![BabyBear::ONE, BabyBear::TWO, BabyBear::ZERO, BabyBear::ZERO]
let evaluations = dft.dft(coeffs);
```

Coset LDE 사용

```
let mat = RowMajorMatrix::new(coeffs, 1);
let lde_result = dft.coset_lde_batch(mat, 2, BabyBear::GENERATOR);
```

Circle FFT 사용

```
use p3_circle::{CircleDomain, CircleEvaluations};
use p3_mersenne_31::Mersenne31;

let domain = CircleDomain::<Mersenne31>::standard(10);
let evals = CircleEvaluations::from_natural_order(domain, trace);
let coeffs = evals.interpolate();
```

보간 사용

```
use p3_interpolation::interpolate_coset;

let point = BabyBear::from_u32(100);
let result = interpolate_coset(&evals_matrix, BabyBear::GENERATOR, point);
```

성능 최적화 팁

1. 알고리즘 선택

- **소규모 변환**: Radix2Dit

- **대규모 변환**: Radix2DitParallel
- **메모리 제약**: Radix2Bowers
- **Mersenne31**: Circle FFT

2. 배치 처리

```
// 단일 변환보다 배치 변환이 효율적
let batch_matrix = RowMajorMatrix::new(data, num_columns);
let result = dft.dft_batch(batch_matrix);
```

3. 메모리 관리

```
// 사전 할당으로 메모리 할당 오버헤드 감소
let mut result = Vec::with_capacity(expected_size);
```

4. Twiddle Factor 재사용

```
// 동일한 크기의 변환을 반복할 때 twiddle factor가 자동으로 메모이제이션됨
let dft = Radix2Dit::default(); // twiddle factor 캐시 유지
```

결론

Plonky3의 다항식 연산과 DFT 구현은 다음과 같은 특징을 가집니다:

1. **다양성**: 여러 FFT 알고리즘으로 다양한 사용 사례 지원

2. **최적화**: 각 알고리즘이 특정 상황에서 최적화됨

3. **병렬성**: 효과적인 병렬 처리로 성능 향상

4. **특화**: Circle FFT 같은 필드별 특화 구현

5. **실용성**: 실제 STARK 증명 시스템에서 검증된 성능

이러한 구현들은 STARK 증명 시스템의 핵심 성능을 결정하는 중요한 구성 요소로, 각각의 특성을 이해하고 적절히 선택하여 사용하는 것이 중요합니다.